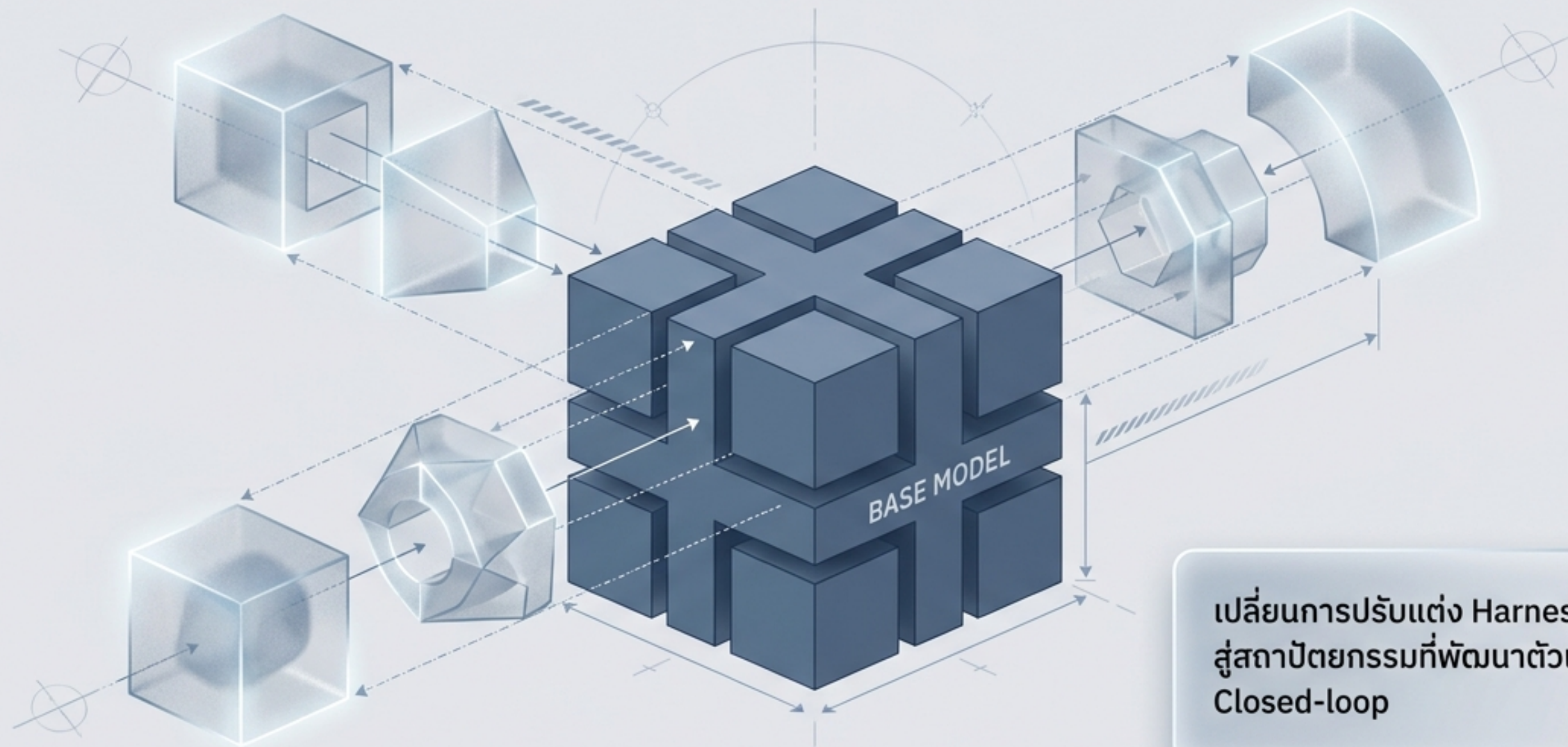


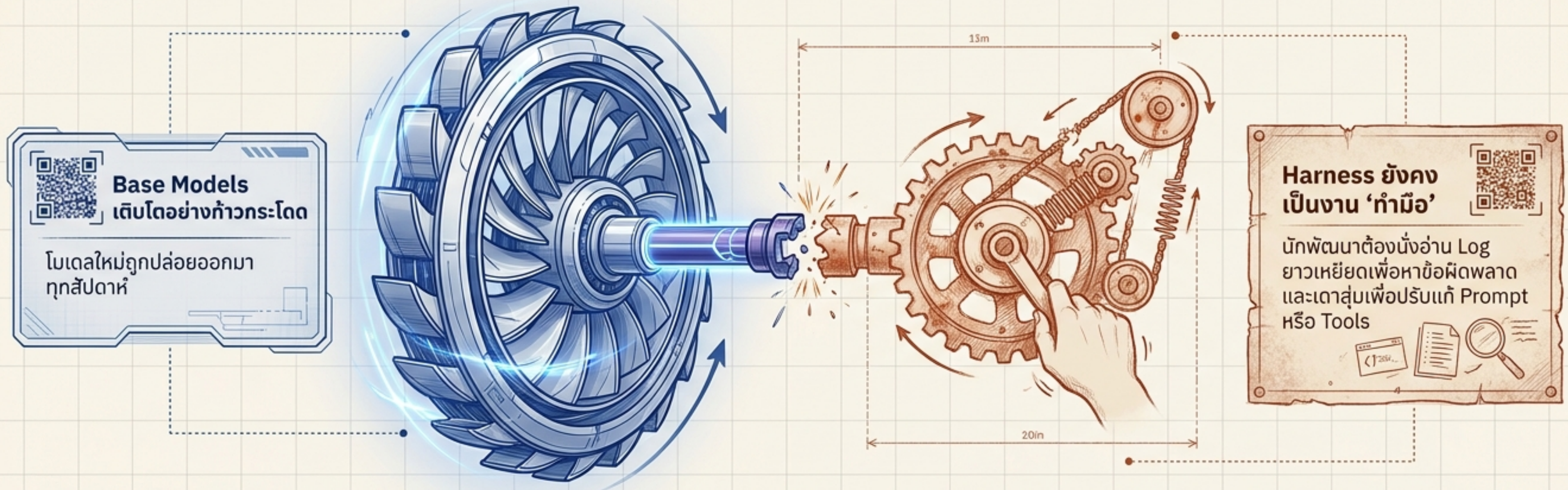
Agentic Harness Engineering (AHE)

ปลดล็อกศักยภาพ Coding Agent ด้วยระบบวิวัฒนาการการอัตโนมัติ



เปลี่ยนการปรับแต่ง Harness ด้วยมนุษย์
สู่สถาปัตยกรรมที่พัฒนาตัวเองได้แบบ
Closed-loop

คอขวดของวงการ AI Agent ในปัจจุบัน



การเชื่อมต่อที่ขาดตอน: เมื่อ Base Model เปลี่ยน Harness เดิมที่คุ้นด้วยมนุษย์มักพังทลายและใช้ด้วยกันไม่ได้

⚠ ปัจจุบัน Harness เป็นตัวแปรสำคัญที่สุดในการตัดสินความสำเร็จของ Long-horizon task ไม่ใช่แค่ความฉลาดของโมเดลอีกต่อไป

แนะนำ AHE: เครื่องยนต์อัตโนมัติที่เปลี่ยนการ ‘เดาสุ่ม’ เป็น ‘ผลลัพธ์’

NexAU Harness:
โครงสร้างแบบแยกส่วน
(Decoupled Substrate)



Coding Agent & Environment:
ทดสอบและสร้าง Raw Trace (~10M tokens)



Agent Debugger:

กลั่นกรองข้อมูลสู่รายงานวิเคราะห์ (~10K tokens)

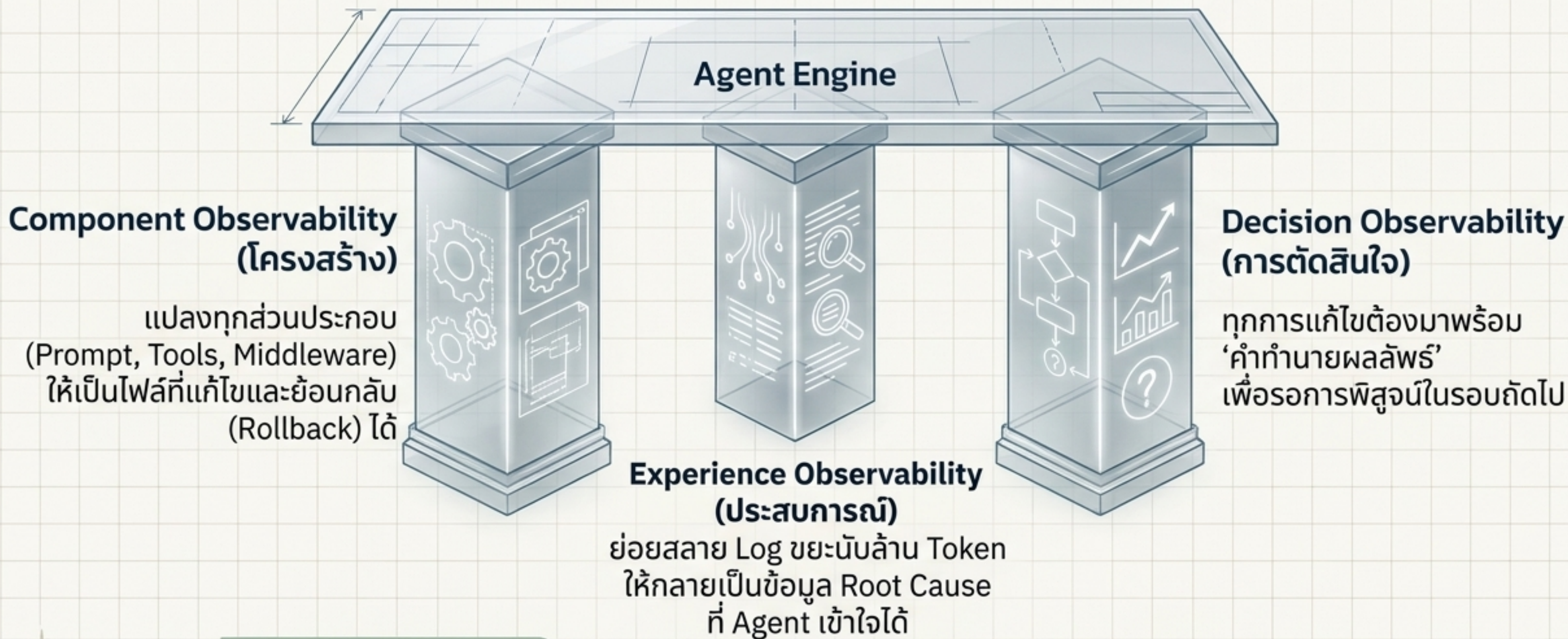


Evolve Agent:
แก้ไขส่วนประกอบอย่างเป็นระบบ



AHE ขับเคลื่อนด้วยโมเดล GPT-5.4 ที่ถูกเซ็ตให้เป็น ‘นักวิจัย’
เพื่อปรับแต่งตัวเองอย่างต่อเนื่องโดยไม่ต้องใช้มนุษย์แทรกแซง

3 เสาหลักแห่ง Observability (ความสามารถในการตรวจสอบได้)



Core Message: "Observability" คือหัวใจที่ทำให้ระบบไม่พังทลายจากการลองผิดลองถูกแบบสุ่ม (Trial-and-error collapse)



Pillar 1: การแยกส่วนสถาปัตยกรรม (NexAU Framework)



System Prompt:
พฤติกรรมและกฎระเบียบ

Tool Description:
คำอธิบายการใช้เครื่องมือ

Tool Implementation:
โค้ดการทำงานของเครื่องมือ

Middleware:
การควบคุม Context
และ Execution

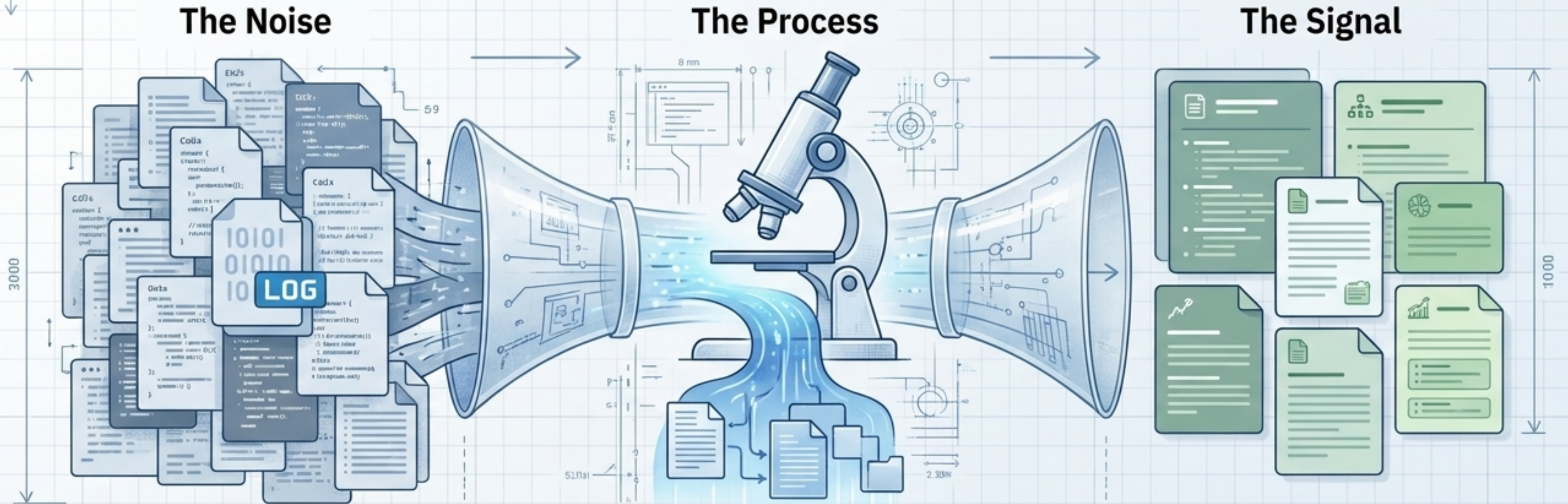
Skills:
แพ็คเกจกระบวนการ
ทำงานซ้ำ

Sub-Agent:
การแบ่งย่อยงาน
ให้เอเจนต์ตัวลูก

Long-Term Memory:
ความทรงจำข้ามเซสชัน

Insight: การแยกส่วนทำให้ AHE
สามารถล็อกเป้าหมายการแก้ปัญหาได้ตรงจุด
(เช่น แก๊ทที่ Tool แทนที่จะยัดทุกอย่างลง Prompt)

Pillar 2: Agent Debugger - สกัดรากเหง้าของปัญหา



Raw Trace
> 10,000,000 Tokens

ข้อมูลดิบที่ยาวเกินกว่าที่ Evolution Agent จะวิเคราะห์ได้ทั้งหมด

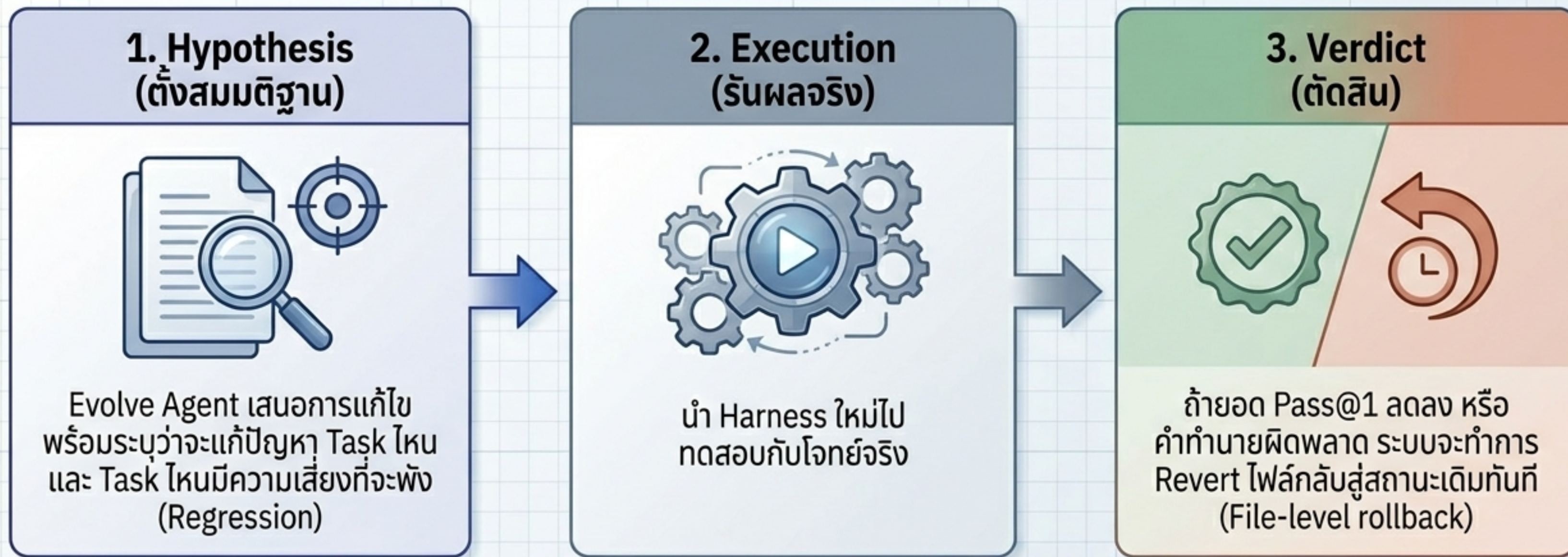
Agent Debugger

ทำการสำรวจเส้นทางการทำงานเหมือนเป็นสภาพแวดล้อมจำลองแบบไฟล์

Layered Evidence Corpus
< 10,000 Tokens

ได้รายงานสรุป, รากเหง้าของปัญหา, และแยกแยะความสำเร็จที่นำไปใช้ต่อได้ทันที

Pillar 3: Manifest Contract - เปลี่ยน 'การแก้ไข' เป็น 'คำสัญญา'

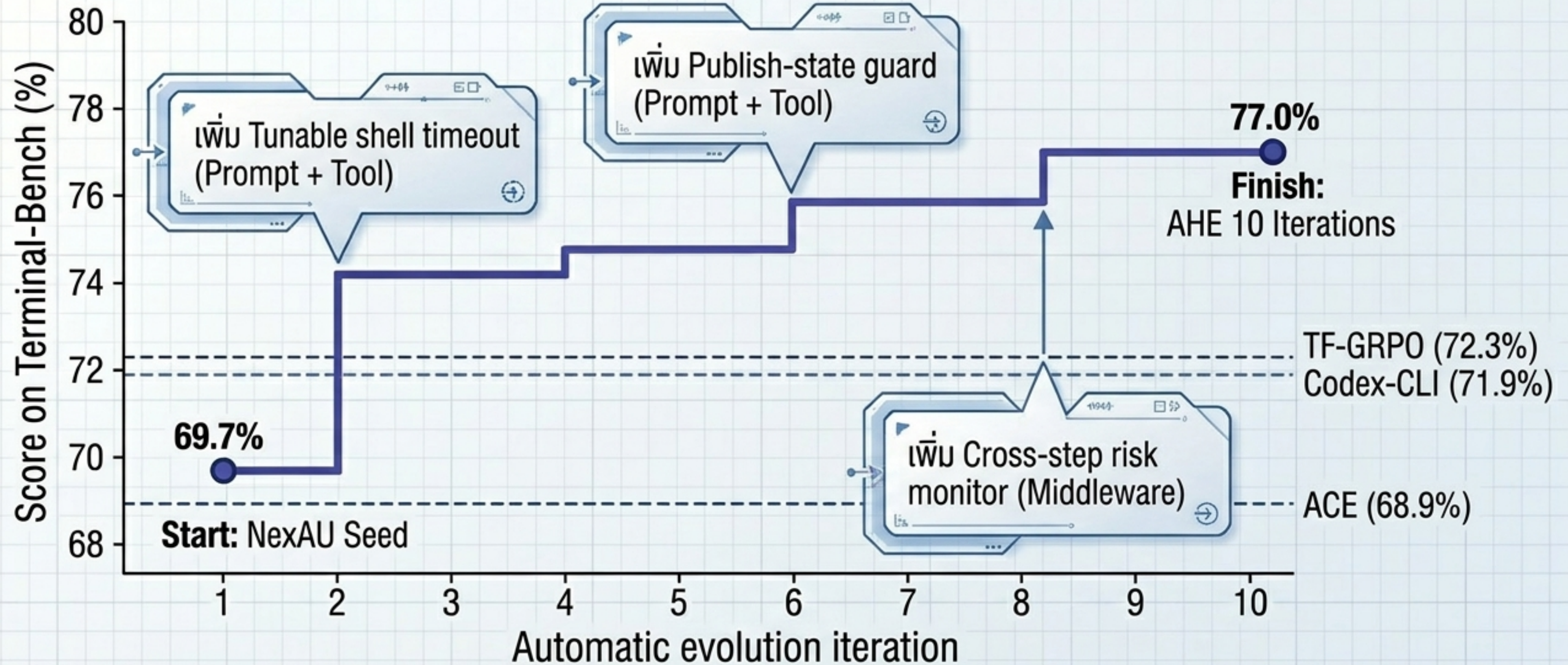


Key Insight: ป้องกันไม่ให้ AI ปรับแต่งตัวเองจนพัง (Self-sabotage) ด้วยกระบวนการแบบ Falsifiable Prediction (การทำนายที่พิสูจน์ความเท็จได้)

The AHE Iteration Loop: วงจรวิวัฒนาการอัตโนมัติ



ผลลัพธ์: ทะลวงขีดจำกัดบน Terminal-Bench 2



ทำไม AHE ถึงเหนือกว่า Self-Evolving AI แบบเดิม?

	ACE ใช้ In-context Playbooks	TF-GRPO Reinforce tool sequences	AHE (Our Method) Full Decoupled Architecture
Natural Language Prompt (การปรับแก้ Prompt)	✓	✓	✓
Tool Execution Logic (การแก้ไขตรรกะเครื่องมือ)	✗	✗	✓
Middleware Integration (การคุม Context และ Execution)	✗	✗	✓
Long-term Memory (ความจำข้ามเซสชัน)	✗	✗	✓

Takeaway: งานวิจัยอื่นพยายามแก้ปัญหาโดยการ ‘เขียน Prompt ให้ยาวขึ้น’
แต่ AHE แก้ปัญหาโดยการ ‘สร้างเครื่องมือและ Middleware ที่ดีขึ้น’

ฆ่าทะเลความสำเร็จ: ชิ้นส่วนใดสร้าง Impact สูงสุด?

+ Long-term Memory +5.6 pp 75.3%

+ Tools +3.3 pp 73.0%

+ Middleware +2.2 pp 71.9%

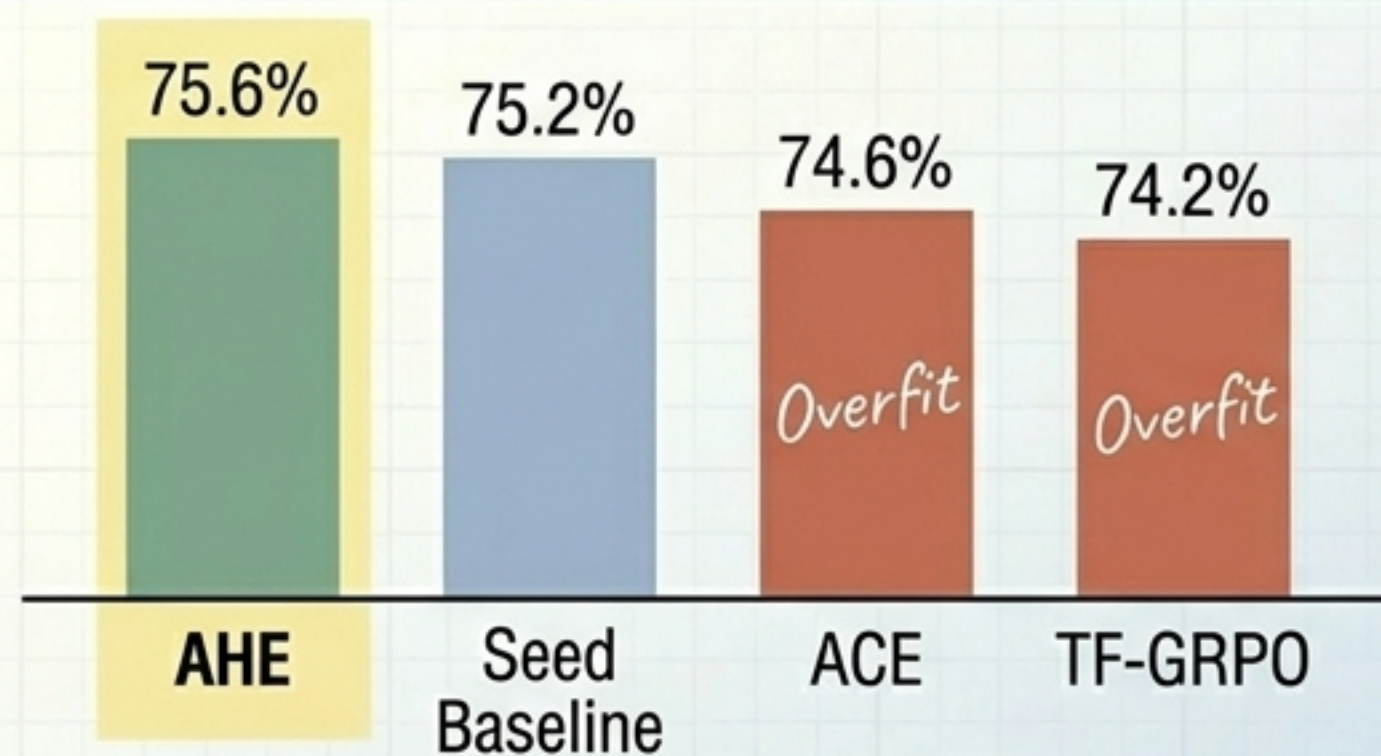
67.4% -2.3 pp + System Prompt อย่างเดียว

Baseline (Seed): 69.7%

Synthesis Insight:
“Prose-level strategy”
(การเขียน Prompt)
ถ่ายทอดและนำไป
ประยุกต์ใช้ซ้ำได้ยาก
ใน ขณะที่
“Factual structure”
(กลไกเครื่องมือและ
หน่วยความจำ)
คือรากฐานที่สร้าง
ความสำเร็จที่แท้จริง

ความสามารถในการทำงานข้ามโดเมน (SWE-bench-verified)

Metric 1: ยอดความสำเร็จ (Success Rate)



AHE ได้คะแนนสูงสุด ชนะ Seed Baseline
ในขณะที่วิธีอื่นทำคะแนนลดลงเพราะ Overfit

Metric 2: ประหยัดค่าใช้จ่าย (Token Efficiency)

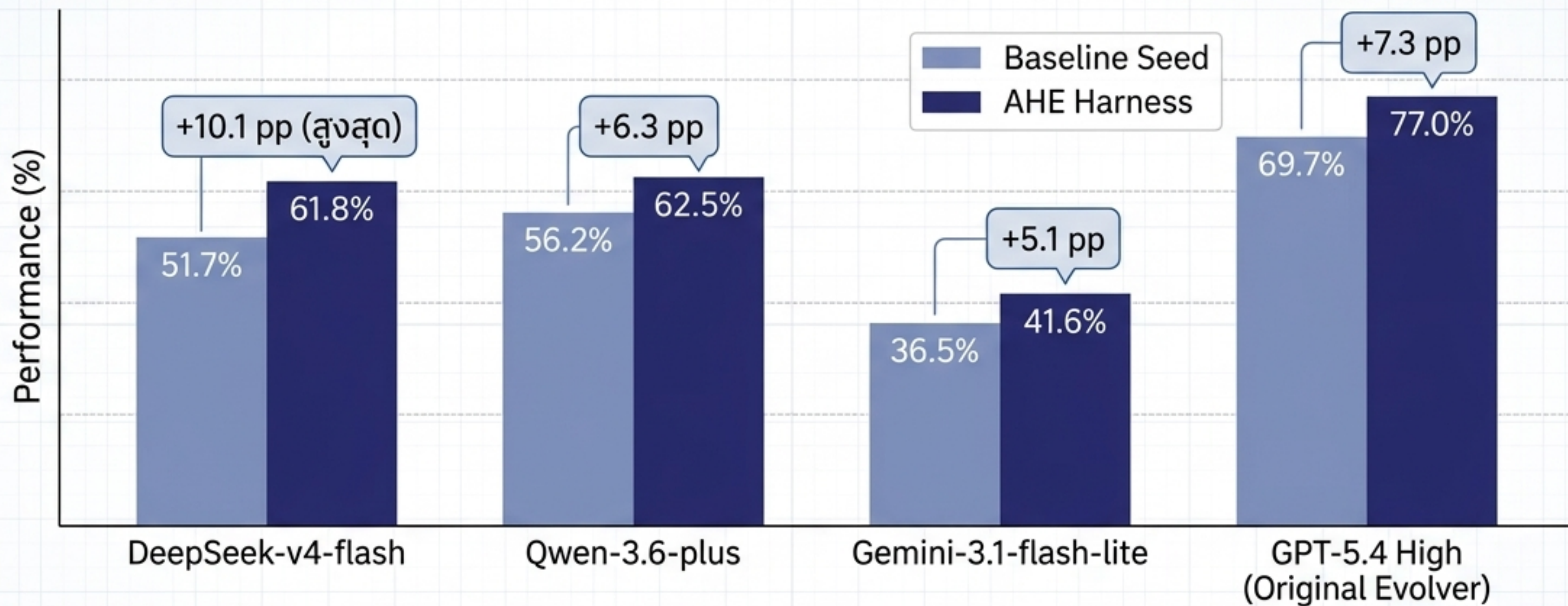
-12%

ลดการใช้ Token ลง 12% เมื่อเทียบกับ Seed Baseline
(และลดถึง 32% เมื่อเทียบกับ ACE)



Insight: การนำความรู้ไปฝังไว้ใน Tool และ Middleware ช่วยประหยัด Token ได้มหาศาลเมื่อเทียบกับการยัดทุกอย่างไว้ใน Prompt ทุกๆ Call

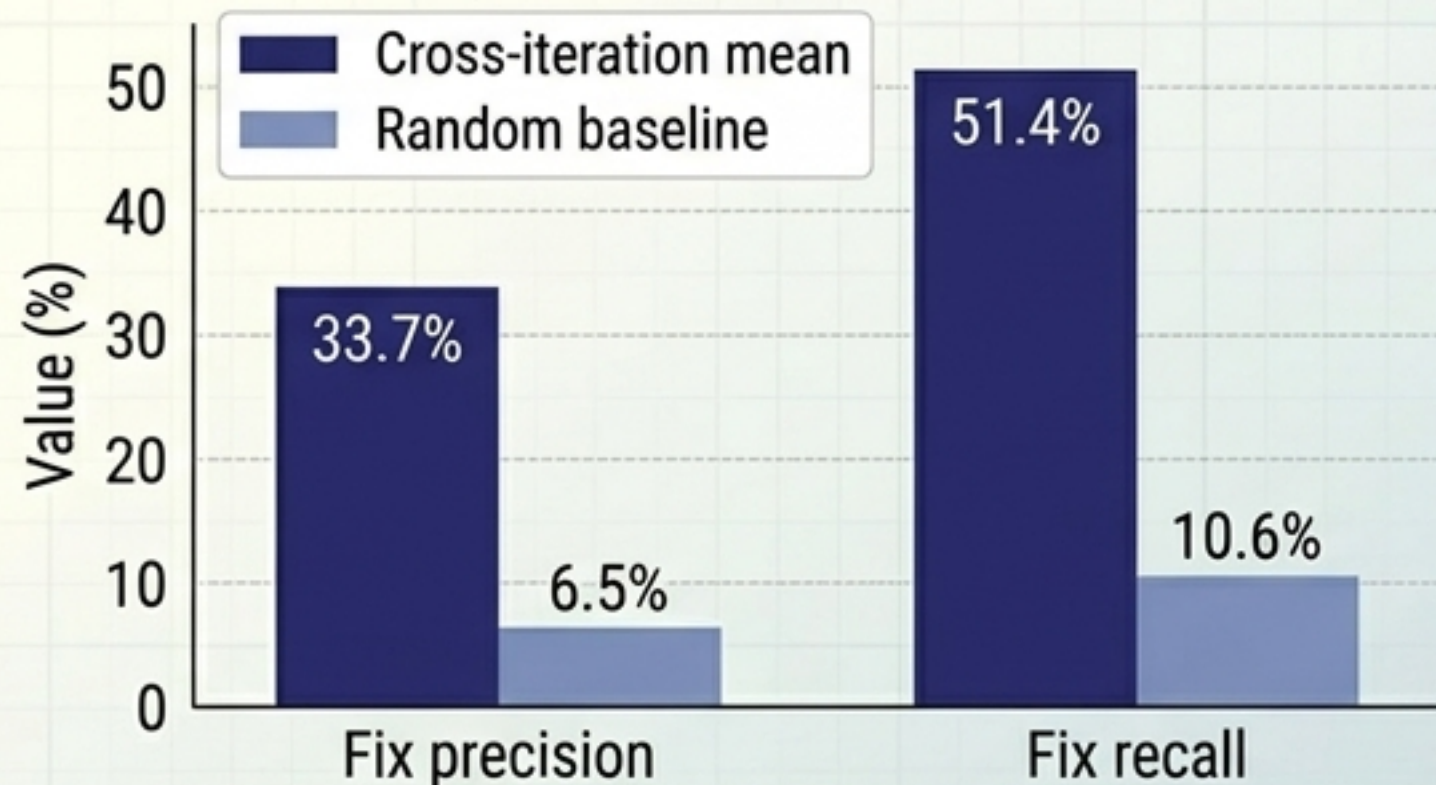
สวม Harness เดียวกัน ให้ Base Model ข้ามค่าย



Key Insight: โมเดลที่ขนาดเล็กกว่าหรือยังไม่ Saturation จะได้รับประโยชน์จาก AHE Harness มากที่สุด ราวกับได้ใส่ ‘ชุดสูทวิศวกรรม’ ที่ถูกปรับแต่งมาอย่างดี

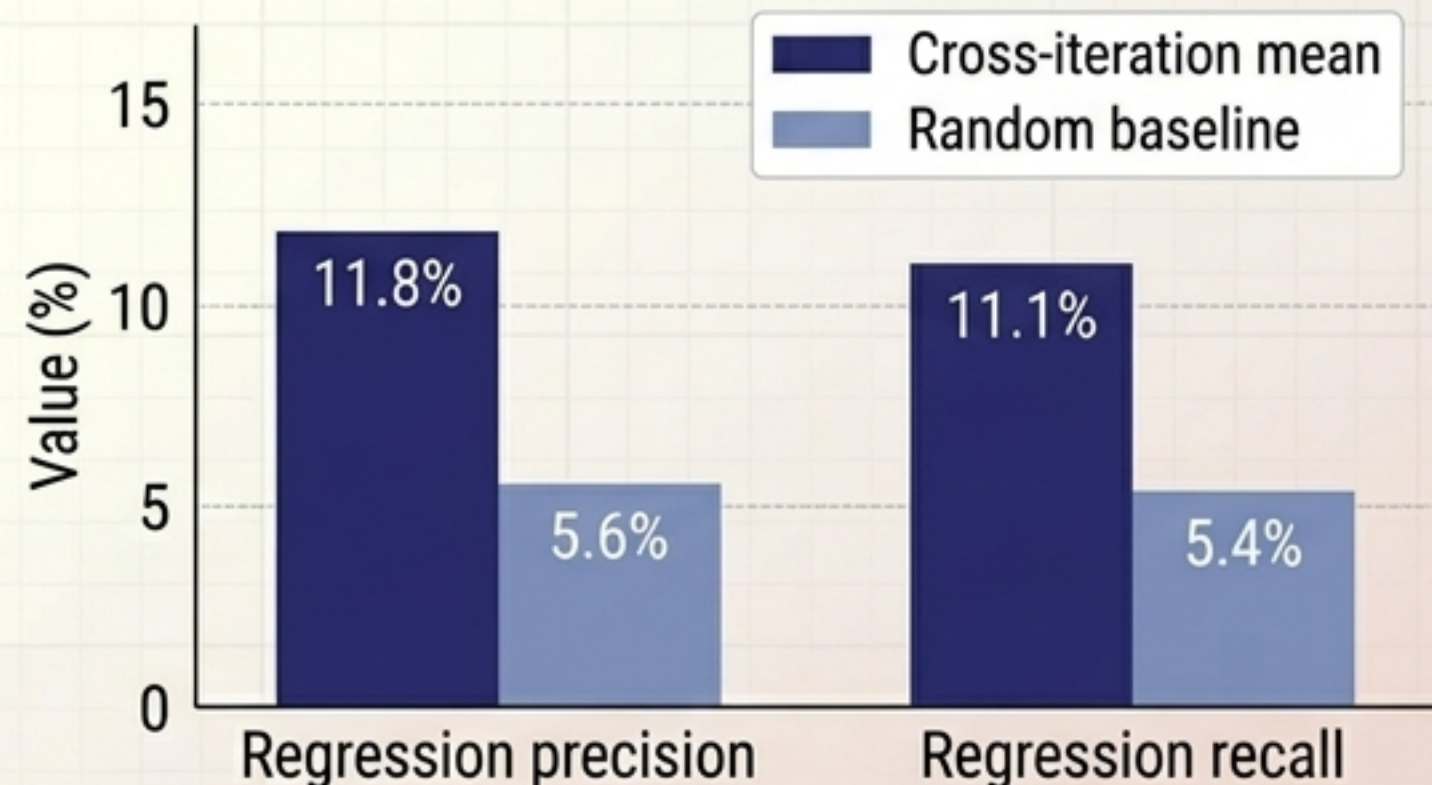
จุดบอดของระบบ: เก่งเรื่อง ‘ซ่อม’ แต่บอดเรื่อง ‘ผลกระทบ’

Evidence-driven targeting (แม่นยำในการแก้ปัญหา)



Agent รู้วิธีแก้บัคเป้าหมายได้อย่างแม่นยำ
(สูงกว่าค่าสุ่มถึง 5 เท่า)

Regression Blindness (ตาบอดผลกระทบวงกว้าง)



แต่ Agent มักดูไม่ออกว่าการแก้โค้ดจุดหนึ่ง
จะไปทำให้ Task อื่นพัง (Regression) ซึ่งเป็นความ
ท้าทายหลักของการวิวัฒนาการตัวเองในอนาคต

บทสรุป: ยุคต่อไปของ AI Agent

1. Stop Prompting, Start Engineering

การพัฒนา Agent ไม่ใช่แค่การเขียน Prompt แต่คือการสร้างชิ้นส่วนสถาปัตยกรรม (Tools, Middleware, Memory) ที่จับต้องได้

2. Observability is the Catalyst

ระบบวิวัฒนาการอัตโนมัติจะเกิดไม่ได้ หากไม่เปลี่ยน Log ชยะให้กลายเป็นโครงสร้างข้อมูลที่ตรวจสอบและ Rollback ได้

3. Cross-Platform Asset

Harness ที่ดีจะกลายเป็น 'ทรัพยากรทางวิศวกรรม' ที่สามารถย้ายไปสวมให้กับโมเดลค่ายใดก็ได้ในอนาคต

Base Model Scaling

Harness Evolution (AHE)

AHE is an externalized, auditable surface where coding-agent experience accumulates.